

澳門大學
UNIVERSIDADE DE MACAU
UNIVERSITY OF MACAU



信息學院
Faculdade de Ciência da Informação e Computação
Faculty of Information Science and Computing

# Modern Techniques

## EMEN 7029 & EMEN 7104 - Special Topic In Electromechanical Engineering / Robotics

Qingbiao Li

University of Macau

Review ..... 3
Dirty Secret of Deep Learning ..... 19
Optimization is Hard ..... 24
Deeper Neural Networks ..... 29
Activation Functions ..... 39
Parameter Initialization ..... 47
Stochastic Gradient Descent ..... 58
Modern Optimization ..... 66
Weight Decay ..... 74
Coding ..... 78

Was youtube video ok?

YouTube translations better than realtime translation?

Next year, should I record and upload all lectures?

# Review

# Review

Many problems in ML can be reduced to **regression** or **classification**

**Regression** asks how many

- How long will I live?
- How much rain will there be tomorrow?
- How far away is this object?

**Classification** asks which one

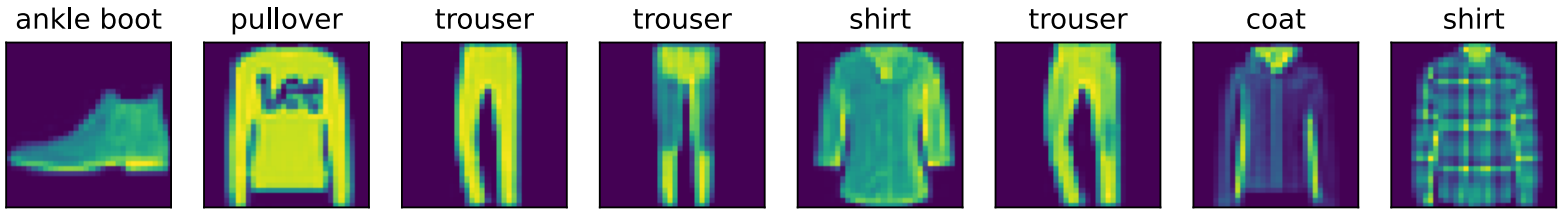
- Is this a dog or muffin?
- Will it rain tomorrow? Yes or no?
- What color is this object?

Last lecture, we defined classification

# Review

**Task:** Given a picture of clothes, predict the text description

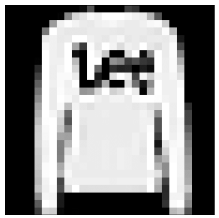
$X : \mathbb{Z}_{0,255}^{32\times32}$




$Y$ : {T-shirt, Trouser, Pullover, Dress, Coat,

Sandal, Shirt, Sneaker, Bag, Ankle boot}

**Approach:** Learn $\boldsymbol{\theta}$ that produce **conditional probabilities**

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = P(\boldsymbol{y} \mid \boldsymbol{x}) = P\left( \begin{bmatrix} \text{T-Shirt} \\ \text{Trouser} \\ \vdots \end{bmatrix} \;\middle|\; \text{  } \right) = \begin{bmatrix} 0.2 \\ 0.01 \\ \vdots \end{bmatrix}$$

# Review

If $B$ provides information about $A$, they are **conditionally dependent**

$$P(A \mid B) \neq P(A)$$

$$P(\text{cloud}) = 0.2, P(\text{rain}) = 0.1$$

$$P(\text{rain} \mid \text{cloud}) = 0.5$$

$$P(A \mid B) = \frac{P(A, B)}{P(B)}$$

Walk outside

$$P(\text{Rain}, \text{Cloud}) = 0.1$$

$$P(\text{Cloud}) = 0.2$$

$$P(\text{Rain} \mid \text{Cloud}) = \frac{0.1}{0.2} = 0.5$$

# Review

How can we represent a probability distribution for a neural network?

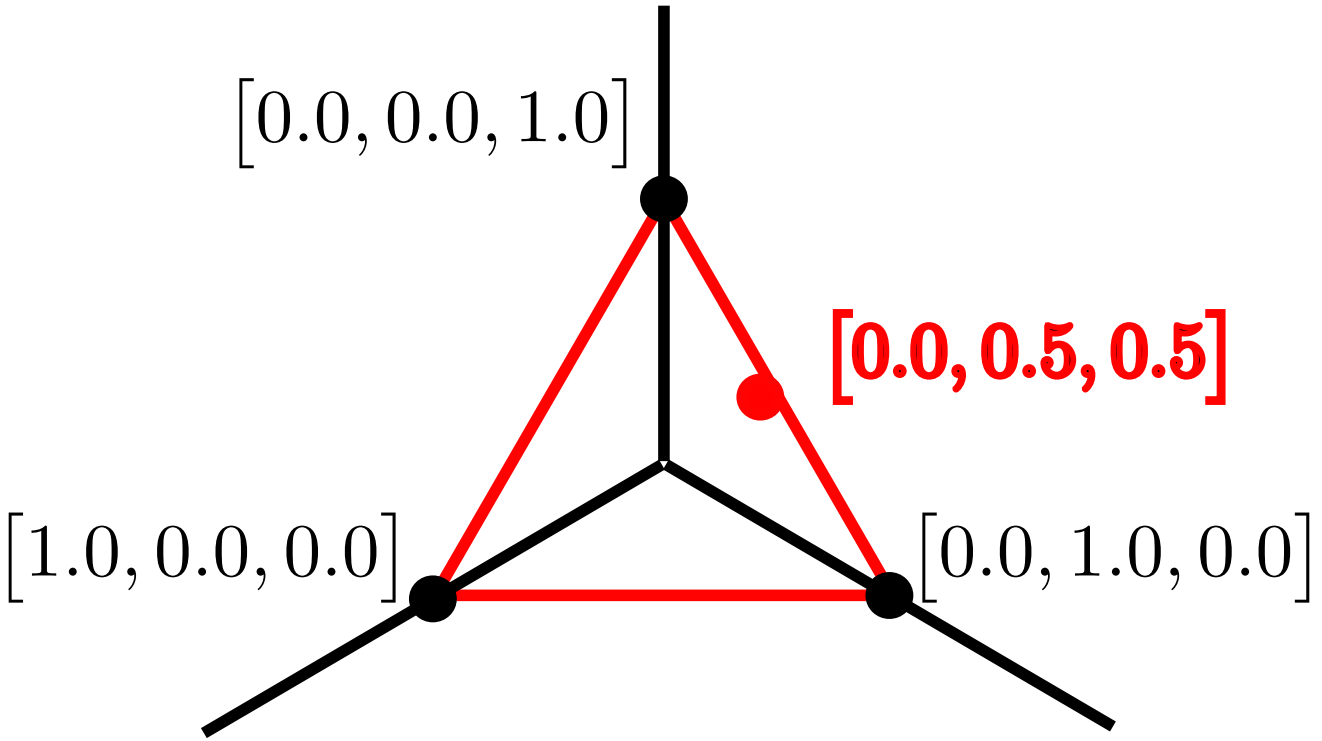
$$\boldsymbol{v} = \left\{ \begin{bmatrix} v_1 \\ \vdots \\ v_{d_y} \end{bmatrix} \;\middle|\; \sum_{i=1}^{d_y} v_i = 1; \quad v_i \in [0, 1] \right\}$$

There is special notation for this vector, called the **simplex**

$$\Delta^{d_y - 1}$$

# Review

The simplex $\Delta^{k-1}$ is an $k-1$-dimensional triangle in $k$-dimensional space



# Review

It has only $k-1$ free variables, because $x_k = 1 - \sum_{i=1}^{k-1} x_i$

# Review

The softmax function maps real numbers to the simplex (probabilities) ||
||

$$\text{softmax} : \mathbb{R}^k \mapsto \Delta^{k-1}$$

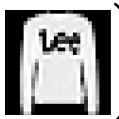
$$\text{softmax}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix}\right) = \frac{e^{\boldsymbol{x}}}{\sum_{i=1}^{k} e^{x_i}} = \begin{bmatrix} \frac{e^{x_1}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \\ \frac{e^{x_2}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \\ \vdots \\ \frac{e^{x_k}}{e^{x_1}+e^{x_2}+\ldots e^{x_k}} \end{bmatrix}$$

If we attach it to our model, we can output probabilities!

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \text{softmax}(\boldsymbol{\theta}^\top \overline{\boldsymbol{x}})$$

# Review

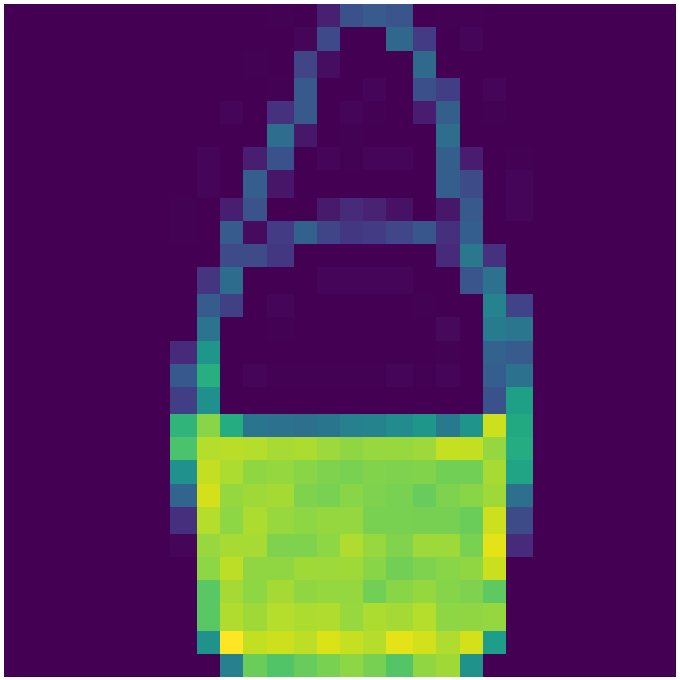
**Question:** Why do we output probabilities instead of a binary values

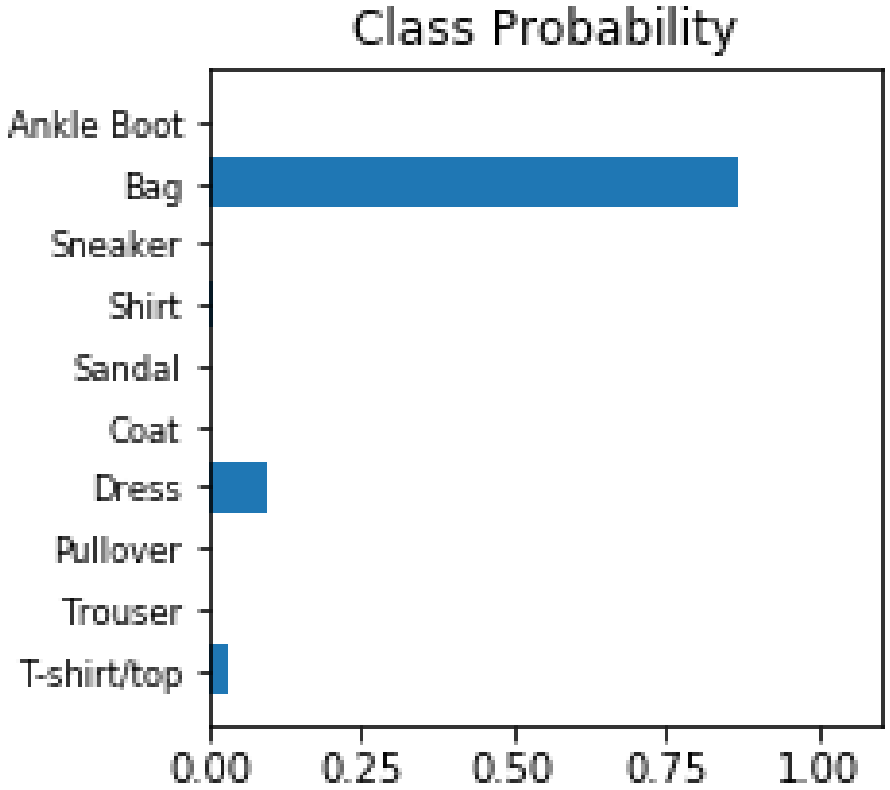
$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} P\Big(\text{Shirt} \mid \text{  }\Big) \\ P\Big(\text{Bag} \mid \text{  }\Big) \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.08 \\ \vdots \end{bmatrix}; \quad f(\boldsymbol{x}, \boldsymbol{\theta}) = \begin{bmatrix} 1 \\ 0 \\ \vdots \end{bmatrix}$$

**Answer 1:** Outputting probabilities results in differentiable functions

**Answer 2:** We report uncertainty, which is useful in many applications

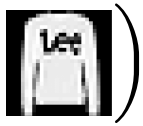
# Review




Class Probability
Ankle Boot
Bag
Sneaker
Shirt
Sandal
Coat
Dress
Pullover
Trouser
T-shirt/top
0.00
0.25
0.50
0.75
1.00

# Review

We consider the label $\boldsymbol{y}_{[i]}$ as a conditional distribution

$$P\left(\boldsymbol{y}_{[i]} \mid \boldsymbol{x}_{[i]}\right) = \begin{bmatrix} P\left(\text{Shirt} \mid \text{  }\right) \\ P\left(\text{Bag} \mid \text{  }\right) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

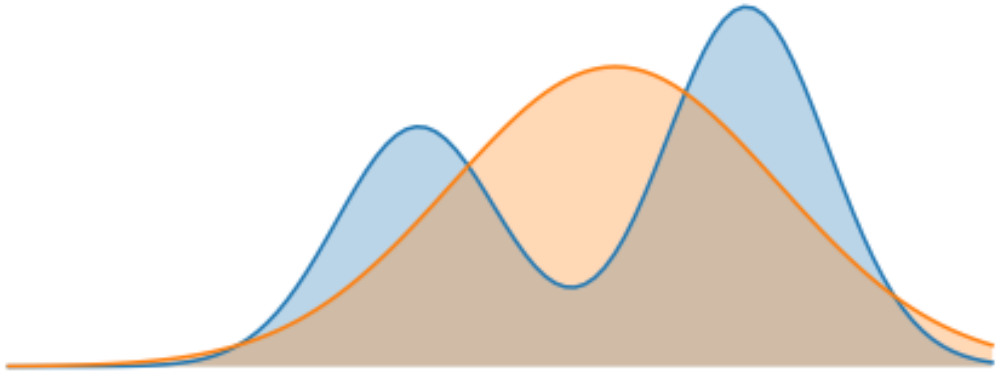
We approximate this with our model $f(\boldsymbol{x}, \boldsymbol{\theta})$

Our loss function measures the difference between two distributions

$$P\left(\boldsymbol{y}_{[i]} \mid \boldsymbol{x}_{[i]}\right), \quad f(\boldsymbol{x}, \boldsymbol{\theta})$$

# Review

The **Kullback-Leibler Divergence (KL)** measures the difference between distributions



# Review

From the KL divergence, we derived the **cross-entropy loss** function, which we use for classification

$$= -\sum_{i=1}^{d_y} P(y_i \mid \boldsymbol{x}) \log f(\boldsymbol{x}, \boldsymbol{\theta})_i$$

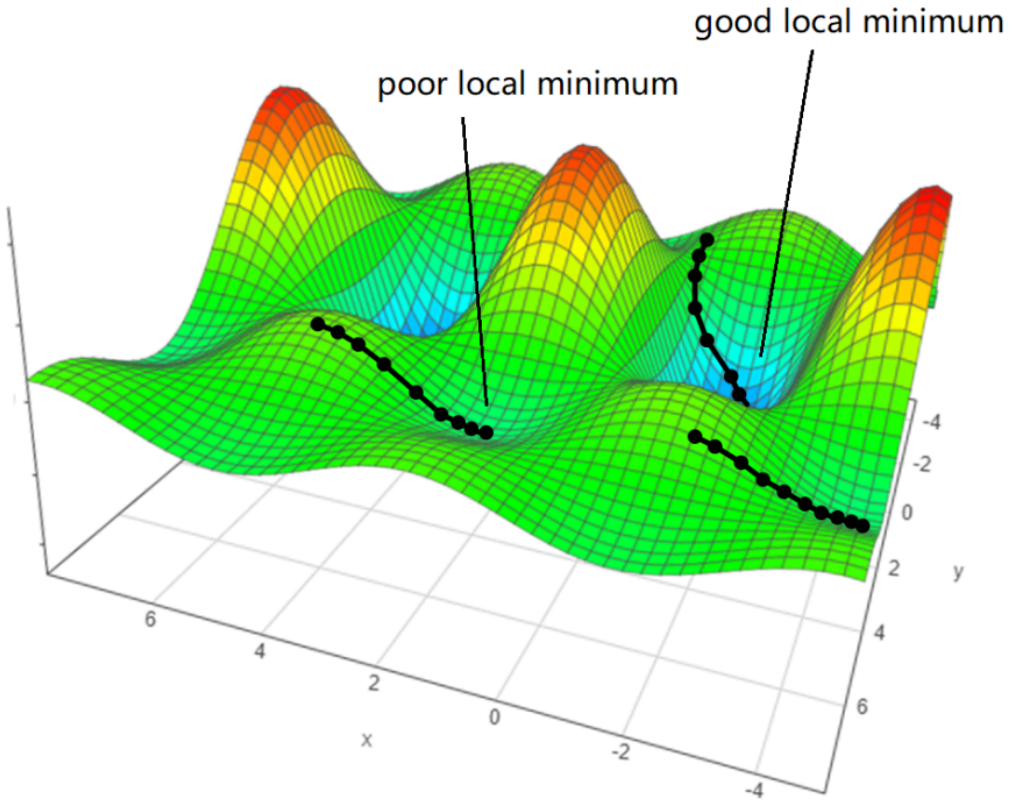
$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{\theta}) = \left[ -\sum_{j=1}^{n} \sum_{i=1}^{d_y} P\left(y_{[j],i} \mid \boldsymbol{x}_{[j]}\right) \log f\left(\boldsymbol{x}_{[j]}, \boldsymbol{\theta}\right)_i \right]$$

# Review

Finish coding exercise

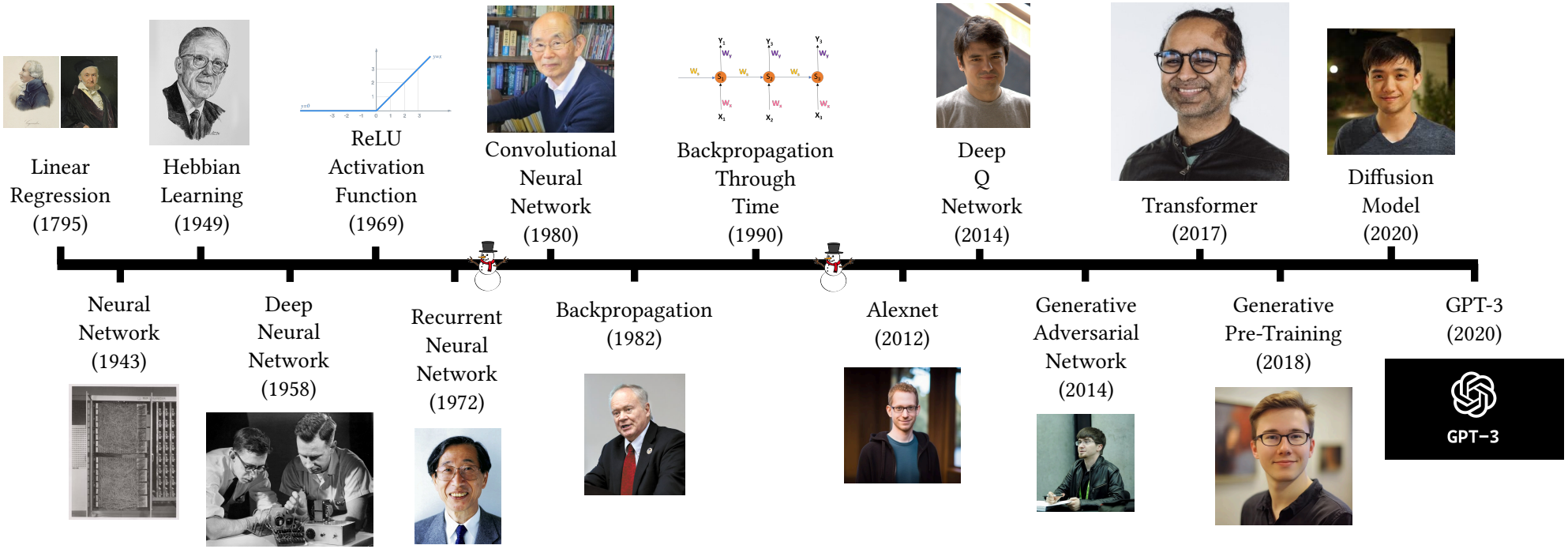
https://colab.research.google.com/drive/16x5yGf8wVhEGY48r0Wgi8uwlsS0AGVC-?usp=sharing

# Review




1. Objective 2. NN architecture 3. Param initialization 4. Optimizer

# Review




Today we will move from 1982 to 2012

# Dirty Secret of Deep Learning

# Dirty Secret of Deep Learning

So far, I gave you the impression that deep learning is rigorous

Biological inspiration, theoretical bounds and mathematical guarantees

For complex neural networks, deep learning is a **science** not **math**

There is no widely-accepted theory for why deep neural networks are so effective

In modern deep learning, we progress using trial and error

- Early DL used deductive reasoning (proofs from axioms)
- Modern DL uses inductive reasoning (hypothesis, experiments)

# Dirty Secret of Deep Learning

Scientific method:

1. Collect observations
2. Form hypothesis
3. Run experiment
4. Publish theory

However, there is a second part of the scientific method:

1. Find existing theory
2. Show counterexample through experiment
3. Publish counterexample
4. Update or falsify theory

Deep learning is new, so much of part 2 has not happened yet!

# Dirty Secret of Deep Learning

For many ideas, the **observations** are stronger than the **theory**

Observe that an idea improves many types of neural networks

Ideas we discuss today are known to improve neural networks

- We will discuss theories about why they improve neural networks
- If you do not believe the theory, prove it wrong and be famous!

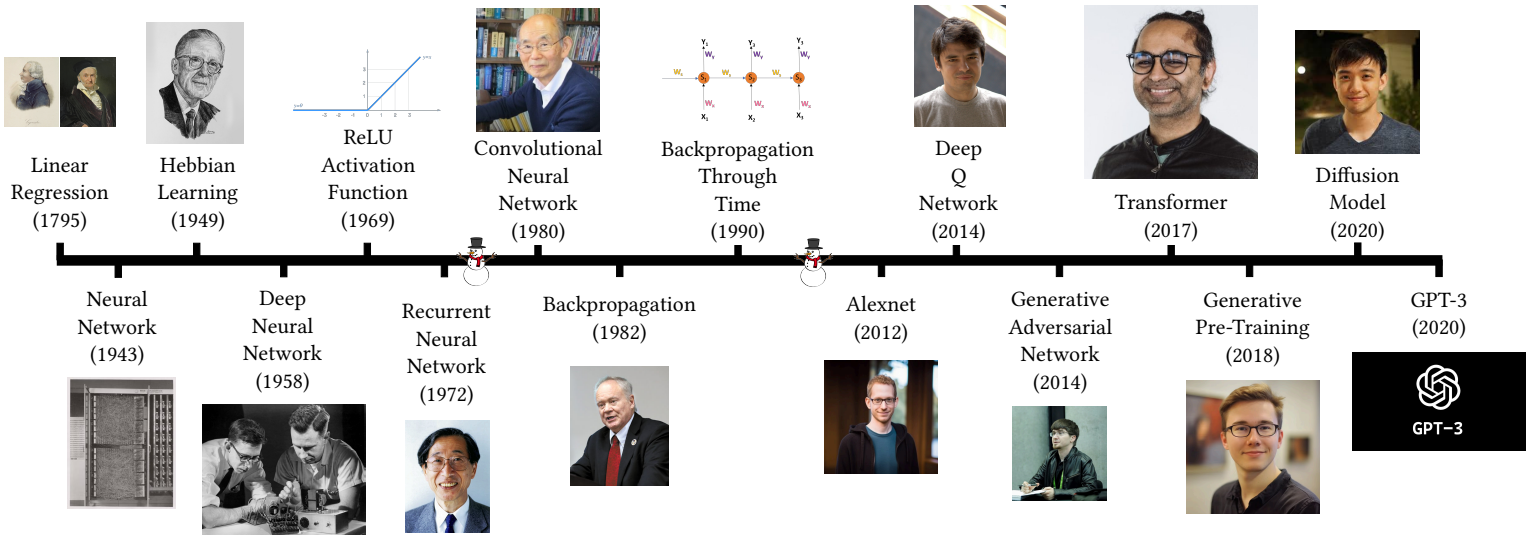
Even if we do not agree on **why** an idea works, if we **observe** that it helps, we can still use it

This is how medicine works (e.g., Anesthetics)!

# Dirty Secret of Deep Learning

Theoretical advancements are still very important!

40 years of neural network research before we understood how to train neural networks




Modern networks too complex for humans to understand

# Optimization is Hard

# Optimization is Hard

A 2-layer neural network can represent **any** continuous function to arbitrary precision

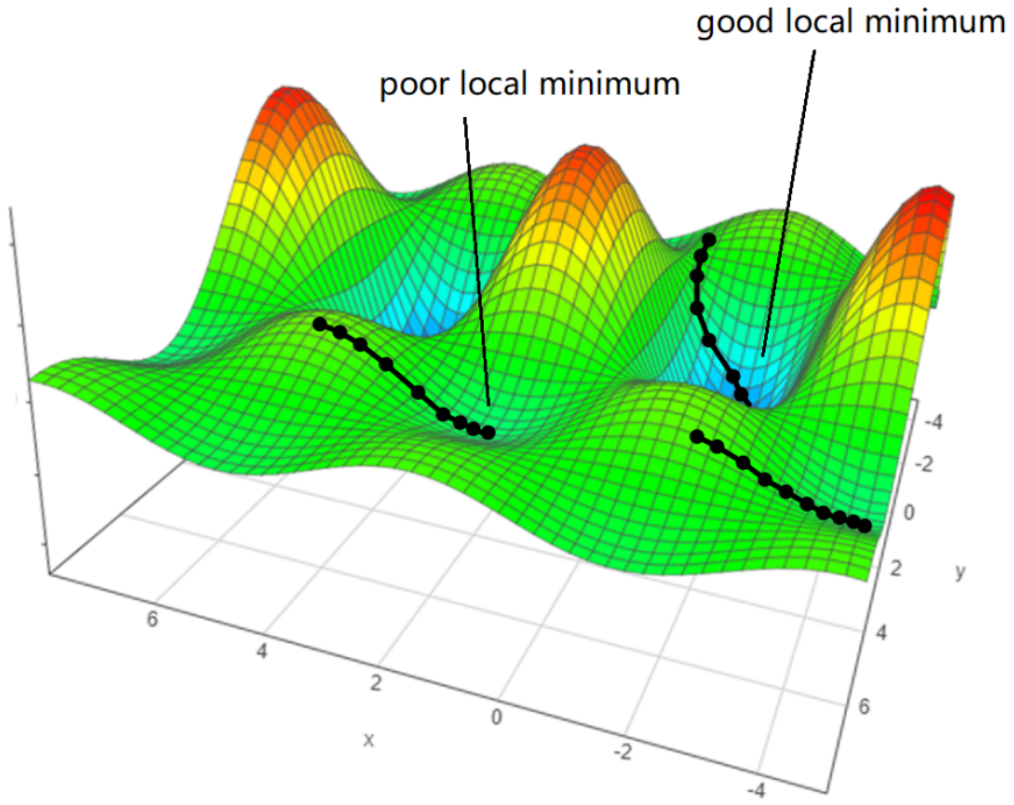
$$| f(\boldsymbol{x}, \boldsymbol{\theta}) - g(\boldsymbol{x}) | < \varepsilon$$

$$\lim_{d_h \to \infty} \varepsilon = 0$$

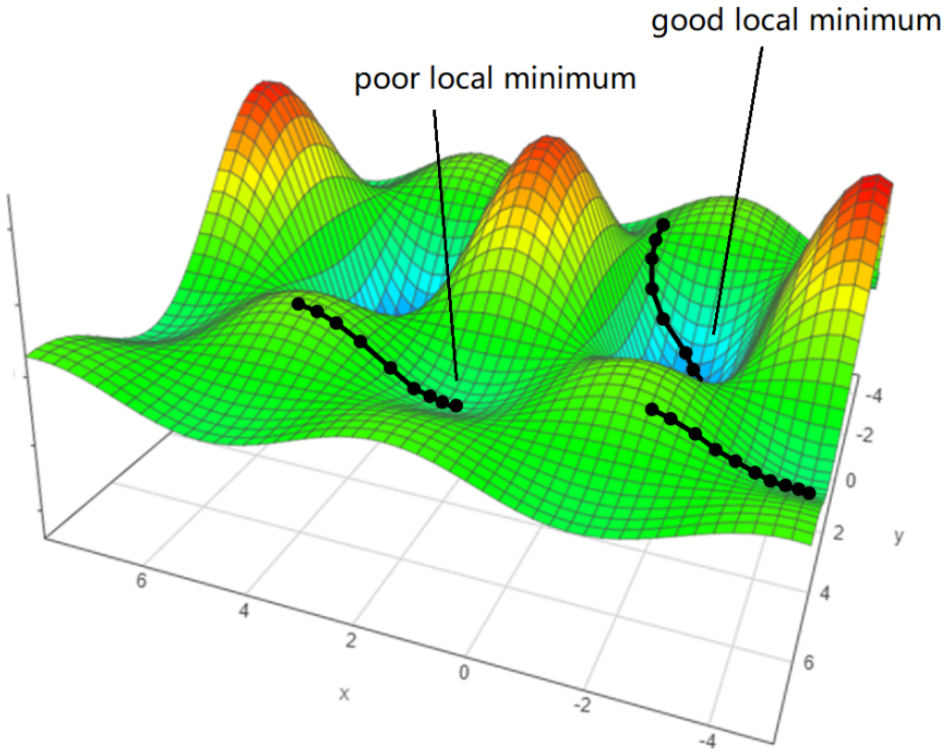
However, finding such $\boldsymbol{\theta}$ is a much harder problem

# Optimization is Hard

Gradient descent only guarantees convergence to a **local** optima



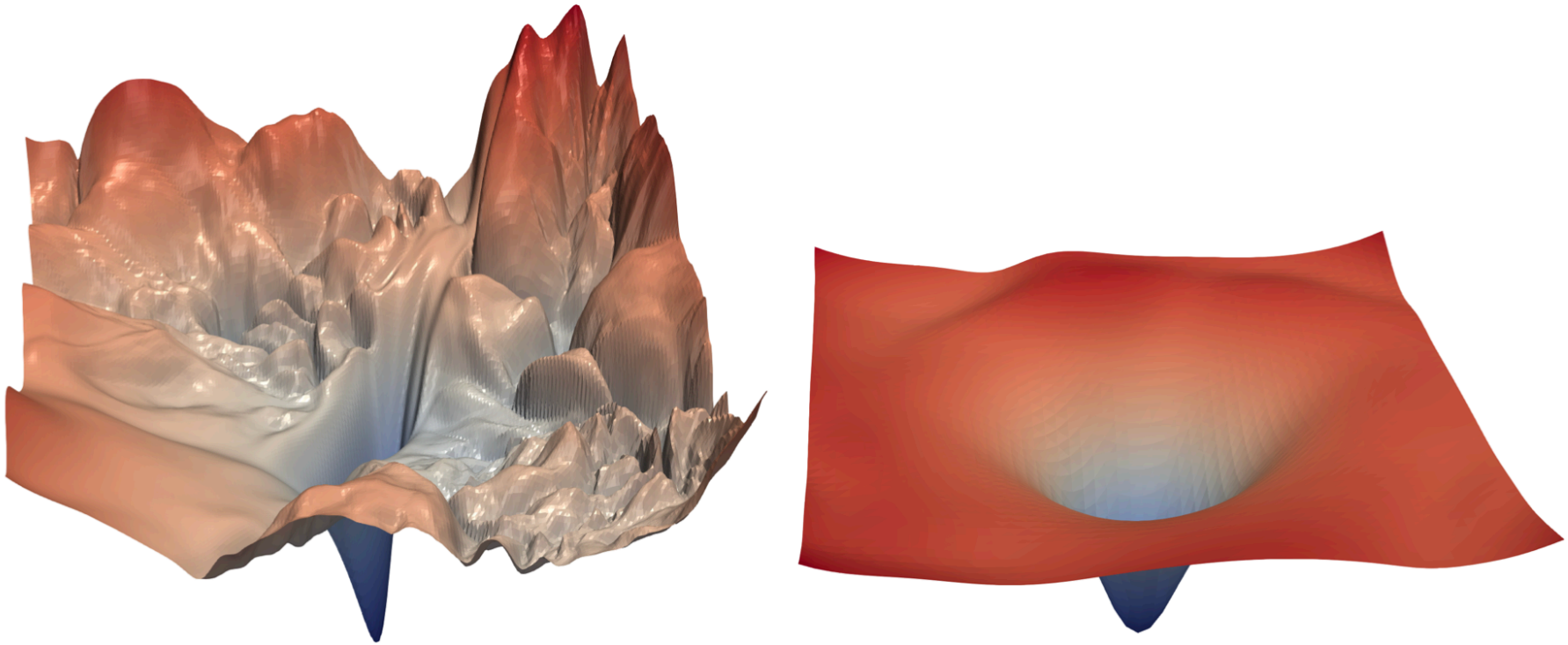
# Optimization is Hard




Harder tasks can have millions of local optima, and many of the local optima are not very good!

# Optimization is Hard

Many of the ideas today create **smoother** loss landscapes



Gradient descent reaches a better optimum more quickly in these cases

# Deeper Neural Networks

# Deeper Neural Networks

A two-layer neural network is sufficient to approximate any continuous function to arbitrary precision

But only with infinite width $d_h \to \infty$

For certain problems, adding one more layer is equivalent to **exponentially** increasing the width

- Eldan, Ronen, and Ohad Shamir. "The power of depth for feedforward neural networks." Conference on learning theory. PMLR, 2016.

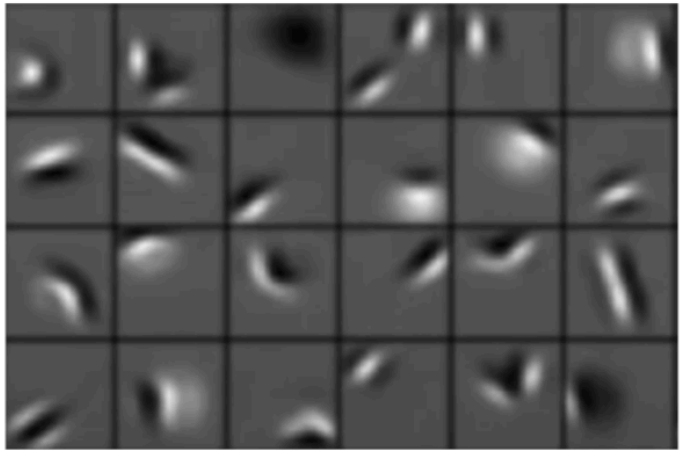
$$256 \times d_y \Rightarrow e^{128} \times d_y \approx 10^{111} \times d_y; \quad \text{universe has } \ 10^{80} \text{ atoms}$$

Another layer can solve this problem in $256 \times 256 + 256 \times d_y$ params

We need more layers for harder problems

# Deeper Neural Networks

Low-level features



Edges, dark spots

Mid-level features



Eyes, ears, nose

High-level features



Facial structure

# Deeper Neural Networks

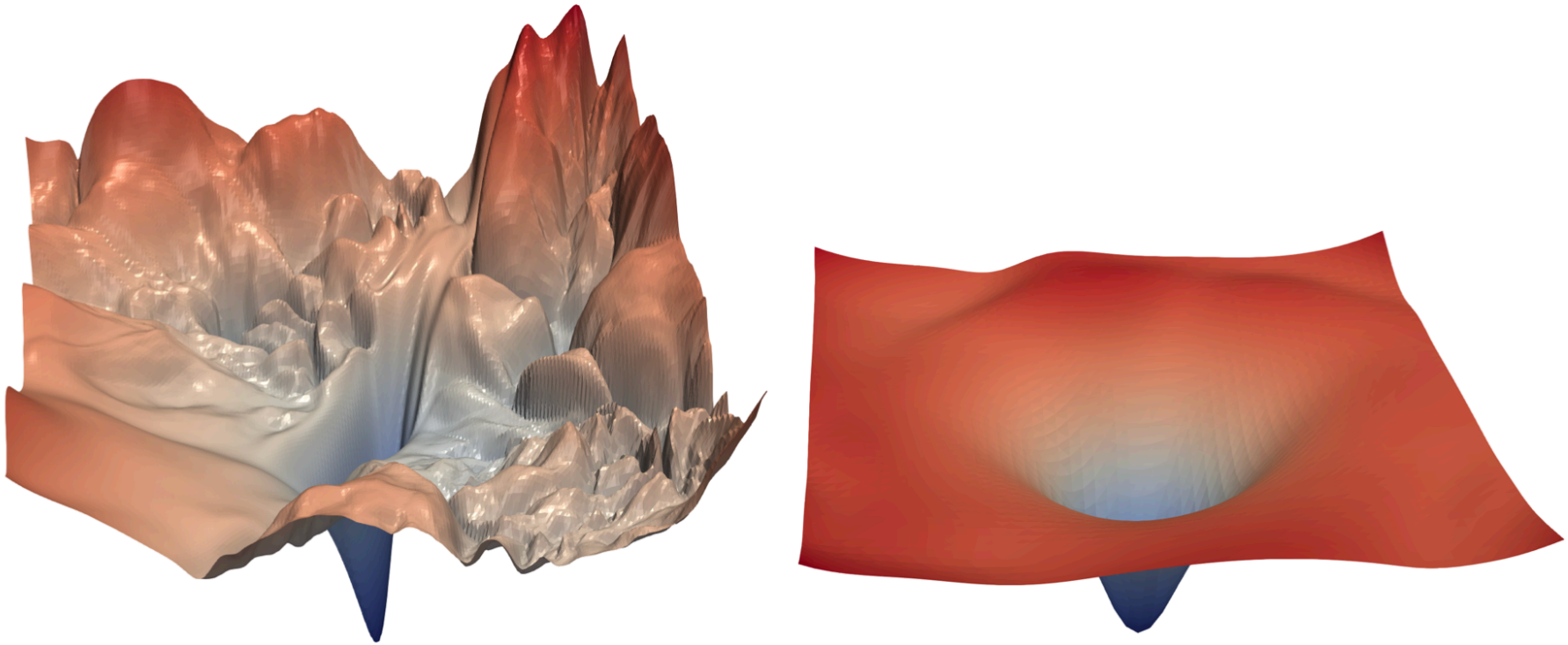
In fact, we do not just need **deeper** networks, but also **wider** networks

The number of neurons in a deep neural network affects the quality of local optima

From Choromanska et al. *The loss surfaces of multilayer networks.* (2014):

- "For large-size networks, most local minima are equivalent and yield similar performance on a test set."
- "The probability of finding a "bad" (high value) local minimum is non-zero for small-size networks and decreases quickly with network size"

# Deeper Neural Networks



# Deeper Neural Networks

This is a difficult finding to conceptualize

The plots I show you are always 3D (2 parameters)

For neural networks with 1,000,000+ parameters, our geometric intuition begins to fail

We understand bigger/deeper networks often perform better, but we are missing deeper theory

We call such networks **overparameterized** neural networks

# Deeper Neural Networks

**Summary:** Deeper and wider neural networks produce better results

Add more layers to your network

Increase the width of each layer

# Deeper Neural Networks

```
# Deep neural network
from torch import nn

d_x, d_y, d_h = 1, 1, 16
# Linear(input, output)
l1 = nn.Linear(d_x, d_h)
l2 = nn.Linear(d_h, d_y)
```

```
# Deeper and wider neural
network
from torch import nn
d_x, d_y, d_h = 1, 1, 256
l1 = nn.Linear(d_x, d_h)
l2 = nn.Linear(d_h, d_h)
l3 = nn.Linear(d_h, d_h)
...
l6 = nn.Linear(d_h, d_y)
```

# Deeper Neural Networks

```
import torch
d_x, d_y, d_h = 1, 1, 256
net = torch.nn.Sequential(
    torch.nn.Linear(d_x, d_h),
    torch.nn.Sigmoid(),
    torch.nn.Linear(d_h, d_h),
    torch.nn.Sigmoid(),
    ...
    torch.nn.Linear(d_h, d_y),
)

x = torch.ones((d_x,))
y = net(x)
```

# Deeper Neural Networks

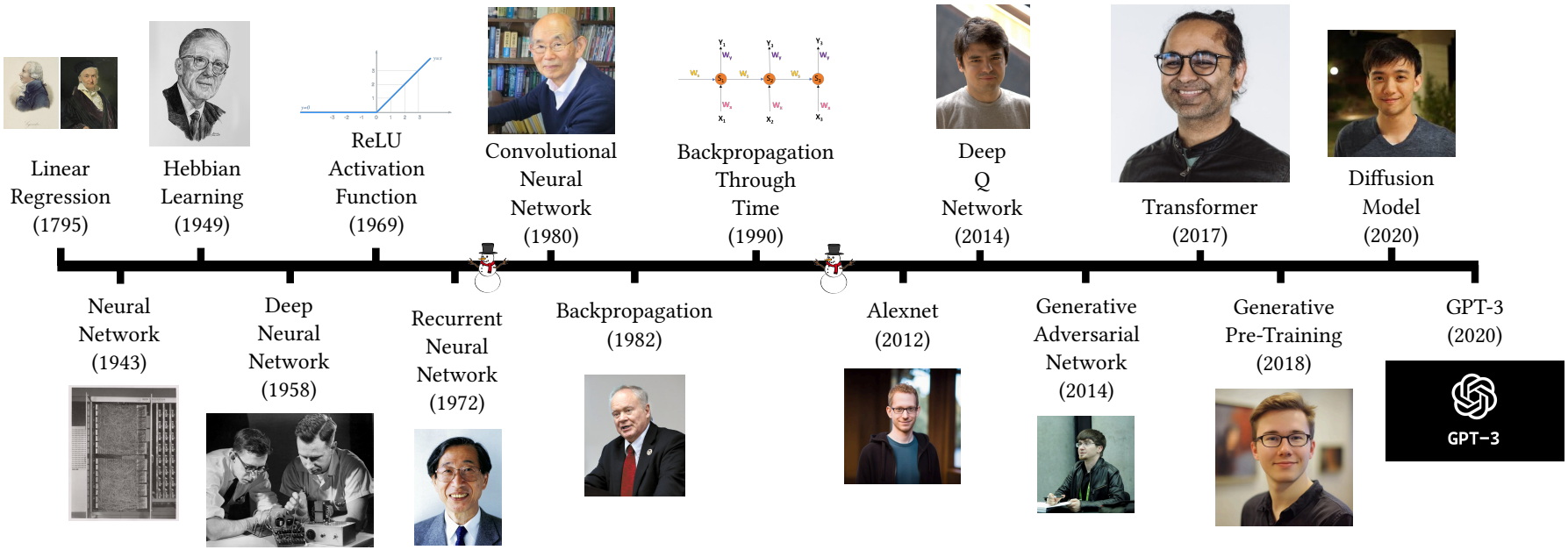
```
import jax, equinox
d_x, d_y, d_h = 1, 1, 256
net = equinox.nn.Sequential([
    equinox.nn.Linear(d_x, d_h),
    equinox.nn.Lambda(jax.nn.sigmoid),
    equinox.nn.Linear(d_h, d_h),
    equinox.nn.Lambda(jax.nn.sigmoid),
    ...
    equinox.nn.Linear(d_h, d_y),
])

x = jax.numpy.ones((d_x,))
y = net(x)
```

# Activation Functions

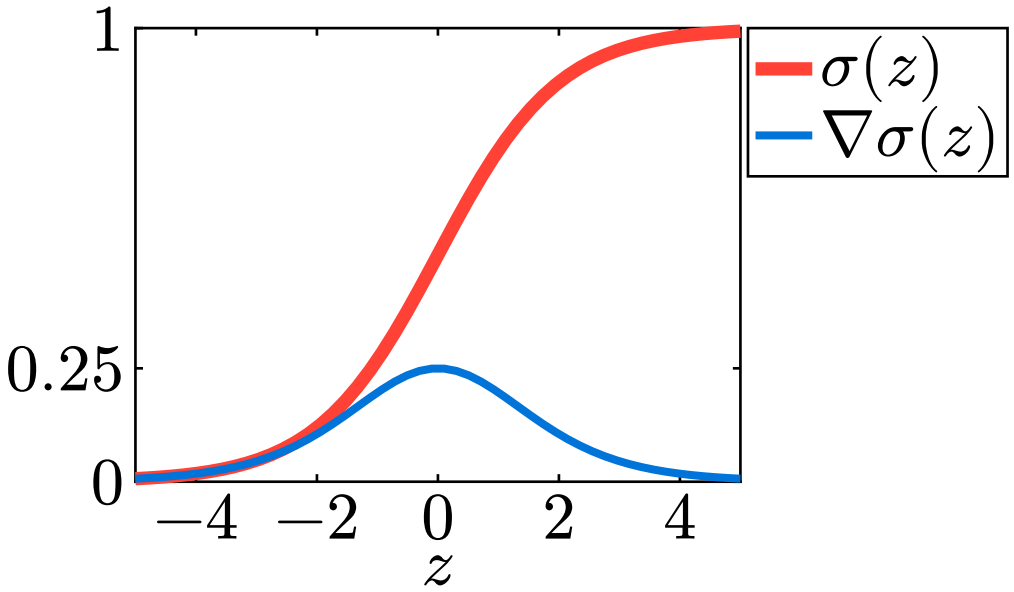
# Activation Functions

The sigmoid function was the standard activation function until ~ 2012




In 2012, people realized that ReLU activation performed much better

# Activation Functions




The sigmoid function can result in a **vanishing gradient**

$$f(\boldsymbol{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}_3^\top \sigma(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}})))$$

$$\left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_3^\top \sigma(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}})))}_{<0.25} \boldsymbol{\theta}_3 \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_2^\top \sigma(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}}))}_{<0.25} \boldsymbol{\theta}_2 \underbrace{(\nabla\sigma)(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}})}_{<0.25} \overline{\boldsymbol{x}}$$

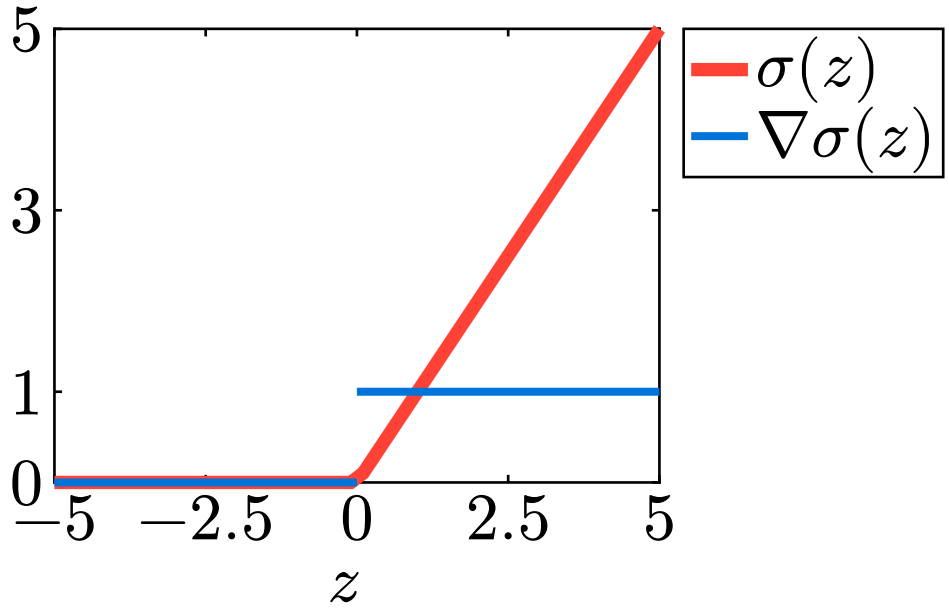
$$\max_{\boldsymbol{\theta}, \boldsymbol{x}} \left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x}, \boldsymbol{\theta}) \approx 0.01 \cdot \ldots$$

$$\lim_{\ell \to \infty} \left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x}, \boldsymbol{\theta}) = 0$$

# Activation Functions

$$\sigma(z) = \max(0, z)$$

$$\nabla\sigma(z) = \begin{cases} 0 \text{ if } z < 0 \\ 1 \text{ if } z \geq 0 \end{cases}$$




**Answer:** Zero gradient region!

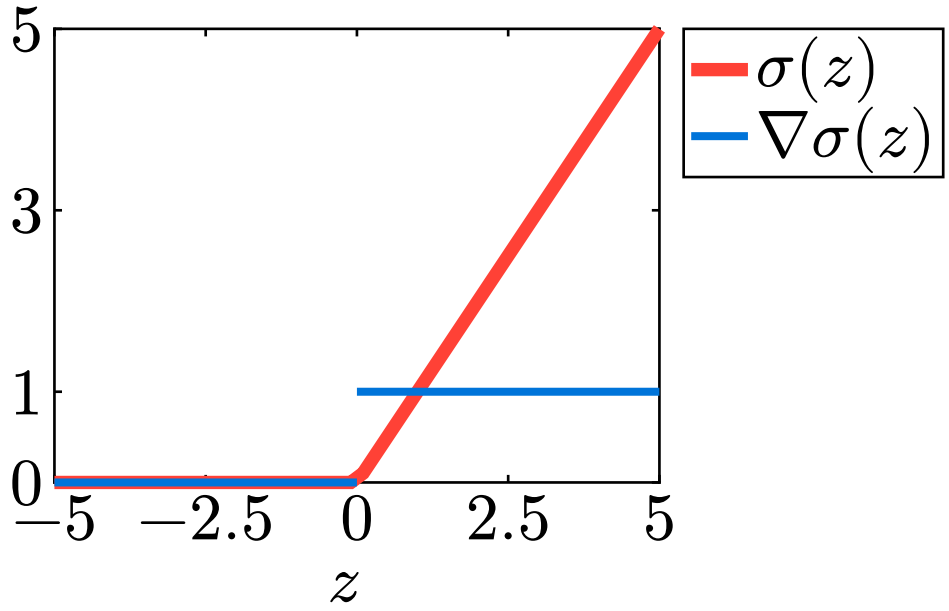
Neurons can get "stuck", always output 0

These neurons cannot recover, they are **dead neurons**

# Activation Functions

$$\sigma(x) = \max(0, x)$$

$$\nabla\sigma(x) = \begin{cases} 0 \text{ if } x < 0 \\ 1 \text{ if } x \geq 0 \end{cases}$$




These neurons cannot recover, they are **dead neurons**

Training for longer results in more dead neurons

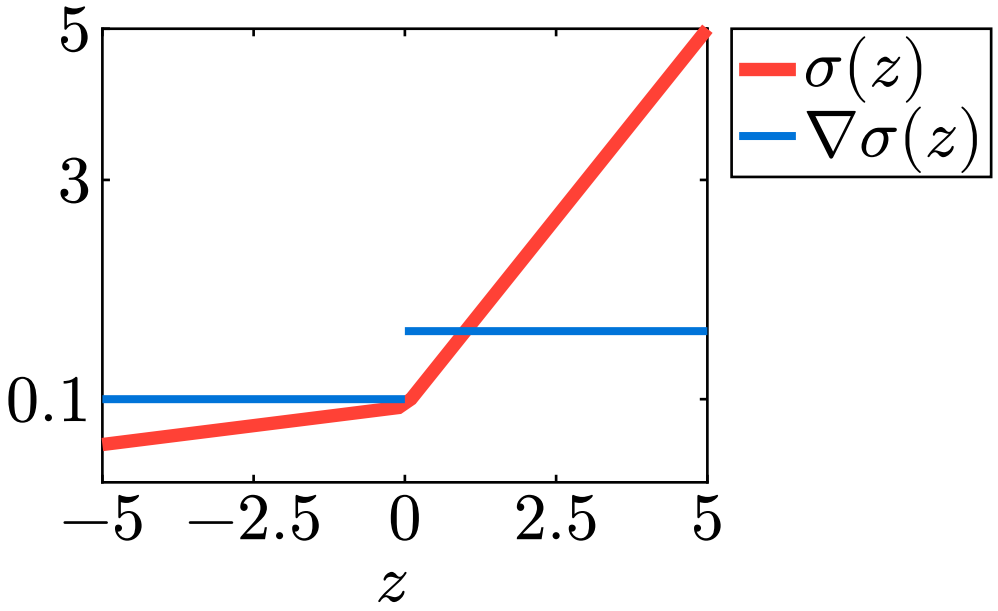
Dead neurons hurt your network (brain damage)!

# Activation Functions

To fix dying neurons, use **leaky ReLU**

$$\sigma(z) = \max(0.1z, z)$$

$$\nabla\sigma(z) = \begin{cases} 0.1 \text{ if } z < 0 \\ 1 \text{ if } z \geq 0 \end{cases}$$



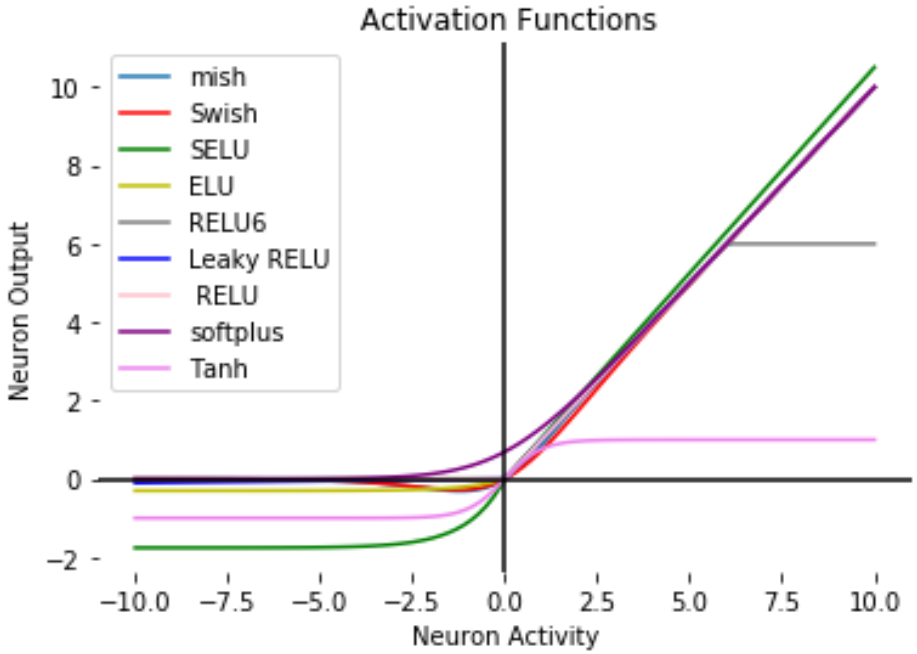

Small negative slope allows dead neurons to recover

As long as one neuron is positive in each layer, non-vanishing gradient

# Activation Functions

There are other activation functions that are better than leaky ReLU

- Mish
- Swish
- ELU
- GeLU
- SeLU




They are all very similar

I usually use leaky ReLU because it works well enough

# Activation Functions

https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity

https://jax.readthedocs.io/en/latest/jax.nn.html#activation-functions

# Parameter Initialization

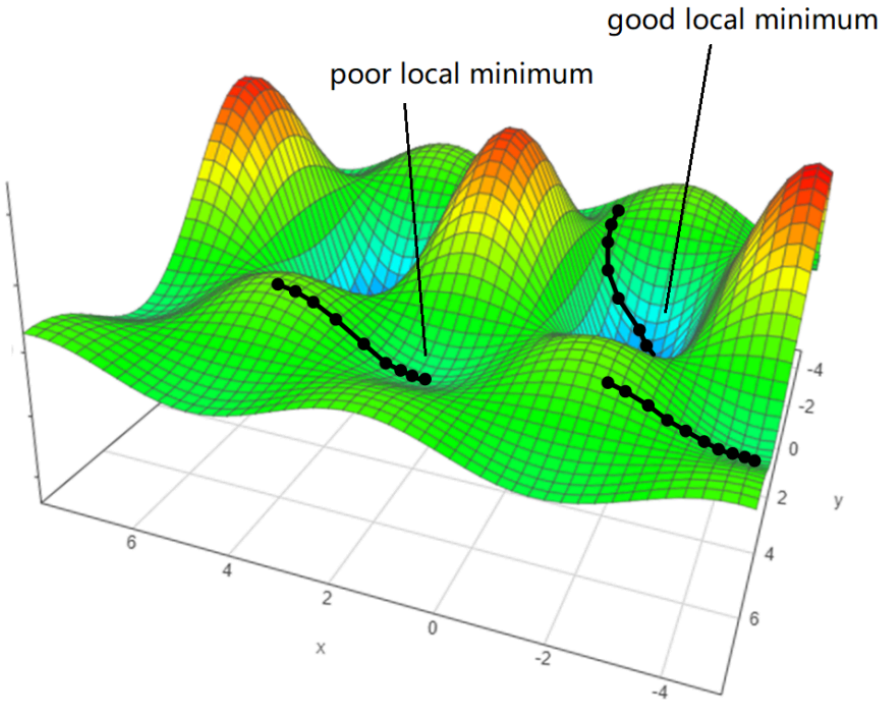
# Parameter Initialization

Recall the gradient descent algorithm

**procedure** GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$)
  $\boldsymbol{\theta} \leftarrow$ Init()
  **for** $i \in \{1...t\}$ **do**
    ▷ Compute the gradient of the loss
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    ▷ Update the parameters using the negative gradient
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
  **end**
  **return** $\boldsymbol{\theta}$
**end**

# Parameter Initialization

Initial $\theta$ is starting position for gradient descent




Pick $\theta$ that results in good local minima

# Parameter Initialization

Start simple, initialize all parameters to 0

$$\boldsymbol{\theta} = \begin{bmatrix} 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{bmatrix}, \begin{bmatrix} 0 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{bmatrix}, \dots$$

**Question:** Any issues?

**Answer:** For ReLU activation, the gradient will always be zero

$$\left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x}, \boldsymbol{\theta}) = (\nabla\sigma)\left(\boldsymbol{\theta}_2^\top \sigma\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}}\right)\right) \cdot (\nabla\sigma)\left(\boldsymbol{\theta}_1^\top \overline{\boldsymbol{x}}\right) \cdot \overline{\boldsymbol{x}}$$

$$\left(\nabla_{\boldsymbol{\theta}_1} f\right)(\boldsymbol{x}, \boldsymbol{\theta}) = \underbrace{(\nabla\sigma)\left(\mathbf{0}^\top \sigma\left(\mathbf{0}^\top \overline{\boldsymbol{x}}\right)\right)}_{0} \cdot \underbrace{(\nabla\sigma)\left(\mathbf{0}^\top \overline{\boldsymbol{x}}\right)}_{0} \cdot \overline{\boldsymbol{x}} = 0$$

# Parameter Initialization

Ok, so initialize $\boldsymbol{\theta} = \mathbf{1}$

$$\boldsymbol{\theta} = \begin{bmatrix} 1 & \dots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 1 \end{bmatrix}, \begin{bmatrix} 1 & \dots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 1 \end{bmatrix}, \dots$$

**Question:** Any issues?

$$z_i = \sigma\left(\sum_{j=1}^{d_x} \theta_j \cdot \overline{x}_j\right) = \sigma\left(\sum_{j=1}^{d_x} \overline{x}_j\right)$$

All neurons in layer have the same gradient,and will receive the same update. Equivalent to a network of width 1, not good!

# Parameter Initialization

We randomly initialize $\boldsymbol{\theta}$ to break symmetry

$$\boldsymbol{\theta} = \begin{bmatrix} -0.5 & \dots & 2 \\ \vdots & \ddots & \vdots \\ 0.1 & \dots & 0.6 \end{bmatrix}, \begin{bmatrix} 1.3 & \dots & 1.2 \\ \vdots & \ddots & \vdots \\ -0.8 & \dots & -1.1 \end{bmatrix}, \dots$$

But what scale? If $\boldsymbol{\theta} \approx 0$ the gradient will vanish to zero, if $|\boldsymbol{\theta}| \gg 0$ the gradient explode to infinity

Almost everyone initializes following a single paper from 2010:

- Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks."
- Maybe there are better options?

# Parameter Initialization

Magic expression, given the input and output size of the layer is $d_h$

$$\boldsymbol{\theta} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{2d_h}}, \frac{\sqrt{6}}{\sqrt{2d_h}}\right]$$

Magic expression with input/output sizes $d_x, d_y$

$$\boldsymbol{\theta} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{d_x + d_y}}, \frac{\sqrt{6}}{\sqrt{d_x + d_y}}\right]$$

# Parameter Initialization

These equations are designed for ReLU and similar activation functions

They prevent vanishing or exploding gradients in deep networks

# Parameter Initialization

Usually `torch` will automatically use Glorot initialization when you create `nn.Linear`

```
layer = nn.Linear(d_x, d_h) # Uses Glorot init
```

You can find many initialization functions at https://pytorch.org/docs/stable/nn.init.html

For JAX it is https://jax.readthedocs.io/en/latest/jax.nn.initializers.html

# Parameter Initialization

```
import torch
d_h = 10
# Manually
theta = torch.zeros((d_h + 1, d_h))
torch.nn.init.xavier_uniform_(theta)
theta = torch.nn.Parameter(theta)

# Using nn.Linear
layer = torch.nn.Linear(d_h, d_h)
# Use .data, to bypass autograd security
torch.nn.init.xavier_uniform_(layer.weight.data)
torch.nn.init.xavier_uniform_(layer.bias.data) # Or zero
```

# Parameter Initialization

Remember, in `torch`, `nn.Linear` is already initialized using Glorot!

In some cases, you may want to use different initializations

- Reinforcement learning
- Recurrent neural networks

# Stochastic Gradient Descent

# Stochastic Gradient Descent

**procedure** GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$)
  $\boldsymbol{\theta} \leftarrow$ Glorot()
  **for** $i \in \{1...t\}$ **do**
    ▷ Compute the gradient of the loss
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    ▷ Update the parameters using the negative gradient
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
  **end**
  **return** $\boldsymbol{\theta}$
**end**

Gradient descent computes $\nabla \mathcal{L}$ over all $\boldsymbol{X}$

# Stochastic Gradient Descent

This works for our small datasets, where $n = 1000$

**Question:** How many GB are the LLM datasets? **Answer:** ~1M GB

This is just the dataset size, the gradient is orders of magnitude larger

$$(\nabla_{\boldsymbol{\theta}} \mathcal{L})\left(\boldsymbol{x}_{[i]}, \boldsymbol{y}_{[i]}, \boldsymbol{\theta}\right) = \overbrace{\begin{bmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & \frac{\partial f_{d_h}}{\partial z_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_{d_h}}{\partial z_1} & \cdots & \frac{\partial f_{d_h}}{\partial z_{d_h}} \end{bmatrix}}^{\text{One layer}}_{[i]} \cdots$$

Gradient at least $O\left(\ell \cdot d_h^2 \cdot n\right) = \underbrace{128 \cdot 16384^2 \cdot 10^{12}}_{\text{GPT guess}} \approx 10^{22} \approx 10^{12}$ GB

# Stochastic Gradient Descent

**Question:** We do not have enough memory to compute the gradient. What can we do?

**Answer:** We approximate the gradient using a subset of the data

We call this **stochastic gradient descent** (SGD)

# Stochastic Gradient Descent

In expectation, the stochastic gradient approximates the true gradient

$$\mathbb{E}_{\boldsymbol{X}_i \sim \boldsymbol{X}, \boldsymbol{Y}_i \sim \boldsymbol{Y}}[(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}_i, \boldsymbol{Y}_i, \boldsymbol{\theta})] = (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$$

**Note:** Only equal under infinitely many samples $\boldsymbol{X}_i, \boldsymbol{Y}_i$

- Approximate expectation using finite samples

$$\hat{\mathbb{E}}_{\boldsymbol{X}_i \sim \boldsymbol{X}, \boldsymbol{Y}_i \sim \boldsymbol{Y}}[(\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}_i, \boldsymbol{Y}_i, \boldsymbol{\theta})] \approx (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$$

# Stochastic Gradient Descent

**procedure** STOCHASTIC GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$, $B$)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  **for** $i \in 1 \ldots t$ **do**
    $\boldsymbol{X}, \boldsymbol{Y} \leftarrow \text{Shuffle}(\boldsymbol{X}), \text{Shuffle}(\boldsymbol{Y})$
    **for** $j \in 0 \ldots \frac{n}{B} - 1$ **do**
      $\boldsymbol{X}_j \leftarrow \begin{bmatrix} \boldsymbol{x}_{[jB]} & \boldsymbol{x}_{[jB+1]} & \cdots & \boldsymbol{x}_{[(j+1)B-1]} \end{bmatrix}^\top$
      $\boldsymbol{Y}_j \leftarrow \begin{bmatrix} \boldsymbol{y}_{[jB]} & \boldsymbol{y}_{[jB+1]} & \cdots & \boldsymbol{y}_{[(j+1)B-1]} \end{bmatrix}^\top$
      $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}_j, \boldsymbol{Y}_j, \boldsymbol{\theta})$
      $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
    **end**
  **end**
  **return** $\boldsymbol{\theta}$
**end**

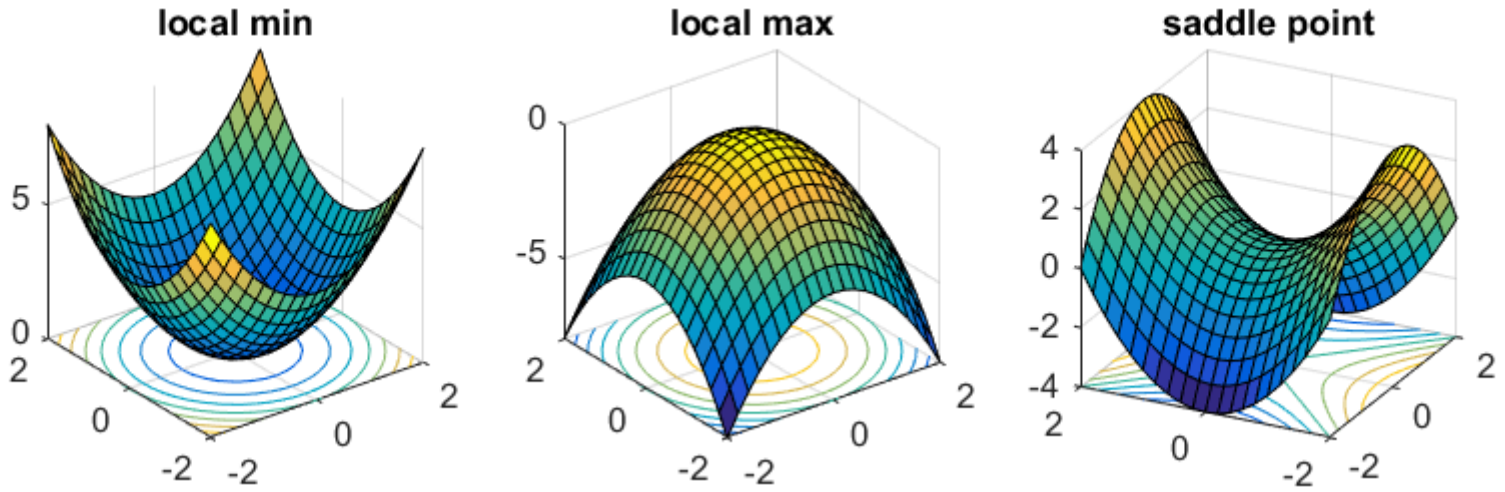
# Stochastic Gradient Descent

Stochastic gradient descent (SGD) is useful for saving memory

But it can also improve performance

Since the “dataset” changes every update, so does the loss manifold

This makes it less likely we get stuck in bad optima



# Stochastic Gradient Descent

`torch.utils.data.DataLoader` helps with SGD

```
import torch
dataloader = torch.utils.data.DataLoader(
    training_data,
    batch_size=32, # How many datapoints to sample
    shuffle=True, # Randomly shuffle each epoch
)
for epoch in number_of_epochs:
    for batch in dataloader:
        X_j, Y_j = batch
        loss = L(X_j, Y_j, theta)
        ...
```

# Modern Optimization

# Modern Optimization

Gradient descent is a powerful tool, but it has issues

1. It can be slow to converge
2. It can get stuck in poor local optima

Improve gradient descent convergence rate, while also preventing premature convergence

It is hard to teach adaptive optimization solely through equations

I want to show you a video to prepare you

https://youtu.be/MD2fYip6QsQ?si=0PFkmaKWKdXAg_Sy&t=197

# Modern Optimization

There are two key concepts behind modern optimizers:

- Momentum
- Adaptive learning rate

Let us discuss the algorithms further

# Modern Optimization

Review gradient descent again, because we will be making changes to it

We usually use SGD (batching), for simplicity I will ignore this

**procedure** GRADIENT DESCENT($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$)
  $\boldsymbol{\theta} \leftarrow$ Glorot()
  **for** $i \in \{1...t\}$ **do**
    ▷ Compute the gradient of the loss
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    ▷ Update the parameters using the negative gradient
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J}$
  **end**
  **return** $\boldsymbol{\theta}$
**end**

# Modern Optimization

Introduce **momentum** first

**function** Momentum Gradient Descent($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$, $\beta$)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{M} \leftarrow \boldsymbol{0}$ # Init momentum
  **for** $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$ # Represents acceleration
    $\boldsymbol{M} \leftarrow \beta \cdot \boldsymbol{M} + (1 - \beta) \cdot \boldsymbol{J}$ # Momentum and acceleration
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{M}$ # Multiply by momentum not gradient
  **end**
  **return** $\boldsymbol{\theta}$
**end**

# Modern Optimization

Now **adaptive learning rate**

**procedure** RMSProp($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$, $\beta$)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{V} \leftarrow \boldsymbol{0}$ # Init variance
  **for** $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$ # Represents acceleration
    $\boldsymbol{V} \leftarrow \beta \cdot \boldsymbol{V} + (1 - \beta) \cdot \boldsymbol{J} \odot \boldsymbol{J}$ # Magnitude
    $\triangleright$ # Rescale grad by magnitude of prev updates
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{J} \oslash \boldsymbol{V}^{\odot \frac{1}{2}}$
  **end**
  **return** $\boldsymbol{\theta}$
**end**

# Modern Optimization

Combine momentum and adaptive learning rate to create **Adam**

**procedure** ADAPTIVE MOMENT ESTIMATION($\boldsymbol{X}$, $\boldsymbol{Y}$, $\mathcal{L}$, $t$, $\alpha$, $\beta_1$, $\beta_2$)
  $\boldsymbol{\theta} \leftarrow \text{Glorot}()$
  $\boldsymbol{M}, \boldsymbol{V} \leftarrow \boldsymbol{0}$
  **for** $i \in 1...t$ **do**
    $\boldsymbol{J} \leftarrow (\nabla_{\boldsymbol{\theta}} \mathcal{L})(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta})$
    $\boldsymbol{M} \leftarrow \beta_1 \cdot \boldsymbol{M} + (1 - \beta_1) \cdot \boldsymbol{J}$ # Update momentum
    $\boldsymbol{V} \leftarrow \beta_2 \cdot \boldsymbol{V} + (1 - \beta_2) \cdot \boldsymbol{J} \odot \boldsymbol{J}$ # Update magnitude
    ▷ # Rescale momentum by magnitude
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \cdot \boldsymbol{M} \oslash \boldsymbol{V}^{\odot \frac{1}{2}}$
  **end**
  **return** $\boldsymbol{\theta}$ # Note, we use biased $\boldsymbol{M}, \boldsymbol{V}$ for clarity
**end**

# Modern Optimization

```
import torch
betas = (0.9, 0.999)
net = ...
theta = net.parameters()

sgd = torch.optim.SGD(theta, lr=alpha)
momentum = torch.optim.SGD(
    theta, lr=alpha, momentum=betas[0])
rmsprop = torch.optim.RMSprop(
    theta, lr=alpha, momentum=betas[1])
adam = torch.optim.Adam(theta, lr=alpha, betas=betas)
...
sgd.step(), momentum.step(), rmsprop.step(), adam.step()
```

# Weight Decay

# Weight Decay

Many modern optimizers also include **weight decay**

Weight decay penalizes large parameters

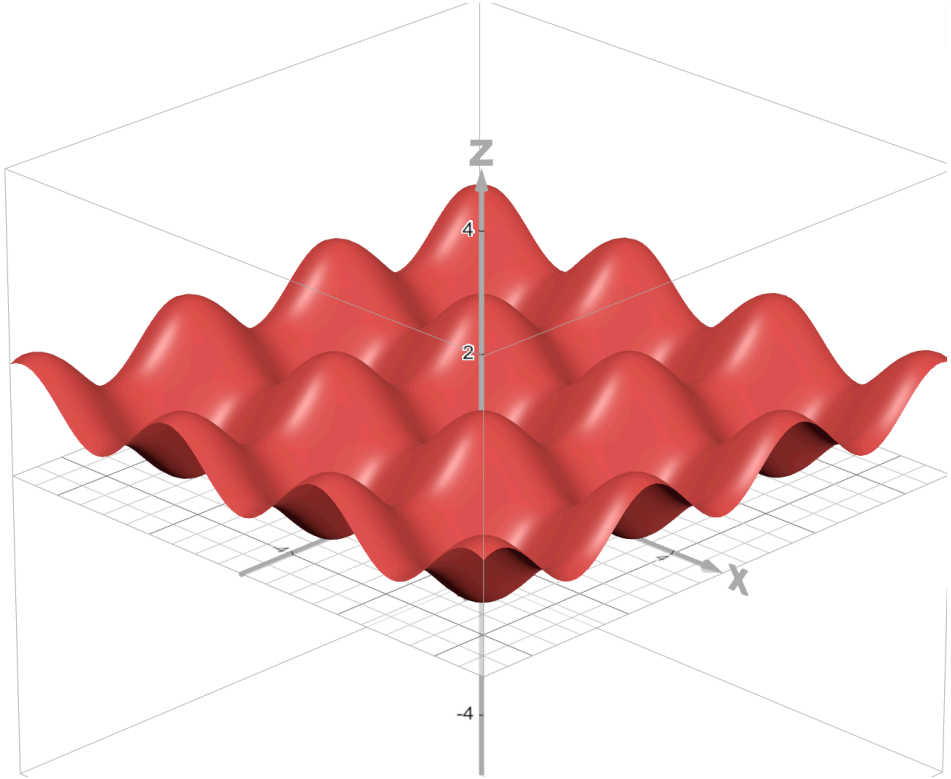
$$\mathcal{L}_{\text{decay}}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) = \mathcal{L}_{\text{decay}}(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\theta}) + \lambda \sum_i \theta_i^2$$

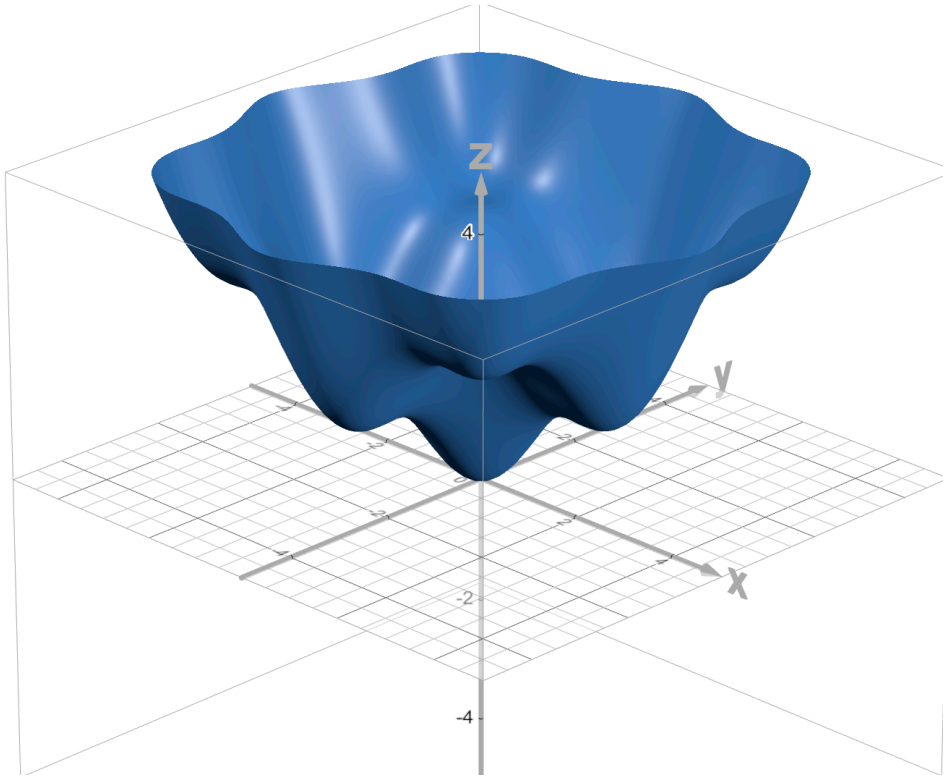
This results in a smoother, parabolic loss landscape that is easier to optimize

(Unproven opinion) It reduces the number of saddle points and local minima

# Weight Decay

$$z = \sin^2(x) + \sin^2(y)$$



# Weight Decay

Weight decay is built into the Adam optimizer

But the implementation is wrong!

Years later, someone noticed and published a new paper that fixes weight decay implementation

```
  Lambda = 0.01
  opt = torch.optim.AdamW(..., weight_decay=Lambda)
//   opt = optax.adamw(..., weight_decay=Lambda)
```

# Coding

# Coding

https://colab.research.google.com/drive/1DFbzclSIeDC306eciMt-mdXDOpYoI67K?usp=sharing